

Vayu-X — Cyclone Intelligence Platform

AI/ML based identification, classification and prediction of tropical cyclone patterns using multi-source satellite data

Problem Statement	PS 26070
Organisation	Ministry of Earth Sciences (MoES) — India Meteorological Department
Theme / Category	Disaster Management · Software
Team	Vayu-X (Team 152) · SIH 2026
Built	10 September 2026

How to use this document. It answers the questions most likely to be asked about the system, in the order they usually come up. Every number here is read directly from the trained model reports at build time, so nothing in it is a rounded-up claim. Where the system is weak, this document says so and says by how much — a limitation you have measured is a stronger answer than one you have not noticed.

1 · What is real, and what is not

The most common question, and the one worth answering first and plainly.

Component	State
Track & intensity forecasting	Real. Trained on IBTrACS best-track data, evaluated against held-out seasons.
Intensity from a satellite image	Real. Trained on 70,257 labelled frames from 494 storms. Held-out MAE 10.25 kt.
Out-of-distribution refusal	Real. Three independent gates; unrelated images are refused, not scored.
Alert rules & message rendering	Real. Rules are data (YAML), tuned without a redeploy.
SMS dispatch	Real (Fast2SMS), operator-triggered only.
Siren tower	Real hardware. ESP32 over USB serial, fires automatically.
Cyclone <i>localisation</i> in a full-disk frame	Not built. Needs INSAT full-disk frames; MOSDAC access pending.
Dvorak <i>pattern</i> classification	Not built. The datasets carry wind speed, not pattern labels. Any pattern shown is inferred from intensity and is labelled as such in the response.
Geofenced delivery to real recipients	Not built. Rules and rendering work; recipient resolution does not.

2 · The map — datasets, storms and timeline

Q. Which four cyclones are on the map, and why those four?

They are chosen by the code, not hand-picked: the four most intense storms in the North Indian Ocean from the seasons the forecasting model was *not* trained on. Selecting by peak intensity keeps the demonstration on the cases that matter, and selecting from held-out seasons keeps it honest.

Storm	Basin	Season	Peak	History shown	Outcome held back
MOCHA	Bay of Bengal	2023	ESCS	41 fixes	10 fixes
TEJ	Arabian Sea	2023	ESCS	24 fixes	19 fixes
BIPARJOY	Arabian Sea	2023	ESCS	49 fixes	58 fixes
KAJIKI	Bay of Bengal	2025	ESCS	14 fixes	11 fixes

Q. Which dataset is the map built from?

IBTrACS v04r01 — the International Best Track Archive for Climate Stewardship, maintained by NOAA/NCEI. For the North Indian Ocean it carries the RSMC New Delhi analyses, which are IMD's own post-storm best-track positions and intensities. It is public, free and requires no credentials. Every storm on the map is a real historical cyclone with its real track.

Q. What is the timeline? What does the map actually show?

Each storm is replayed and paused at its peak intensity. Everything before that moment is given to the model as observed history; everything after is withheld. The model then forecasts forward, and the dashboard draws the forecast track against the *actual* outcome it never saw.

This is why the map shows three different lines: the observed track (coloured by intensity category), the forecast track (dashed) and the actual outcome (solid). A forecast you cannot check is a claim; drawn beside the outcome it is a result.

Q. Which seasons train the model, and which are held out?

The track model trains on seasons **2012–2022** and is tested on **2023–2025**. The map draws only from the test window, so no storm on screen is one the model has memorised.

This is enforced by a test, not by convention. An earlier version replayed from 2021 onward, which put TAUKTAE (2021) on the map while the model had trained through 2022 — its forecast was being scored against data it had already seen. That is now caught automatically if the two windows ever drift apart again.

Q. Is any of the data synthetic or faked for the demo?

No. There is a synthetic generator in the codebase, but it exists only as a fallback so a fresh checkout runs before the archive is downloaded. When it is active the API reports source: `synthetic` and the interface shows a DEMO DATA badge. On the running system it is off — the badge reads REAL DATA.

Q. Are you using INSAT-3D / 3DR, as the problem statement asks?

Not yet, and this is the honest gap. MOSDAC credentials have been applied for and not yet granted. The system is built so that INSAT slots in without redesign: the ingest layer is source-agnostic, and the image model reads infrared structure rather than anything sensor-specific.

What is used instead is geostationary infrared from other satellites — the same physical measurement (cloud-top brightness temperature at $\sim 10.8\ \mu\text{m}$) from a different platform. Expect a domain gap on INSAT frames until the model is retrained on them; the API states this in every response rather than hiding it.

3 · The image upload model

Q. What happens when I upload a satellite image?

The image is sent to the model service, which extracts **108 features** describing the storm's infrared structure — statistics over 16 concentric rings around the centre, asymmetry between quadrants, gradient and texture measures, and an explicit eye signature. A gradient-boosted regressor maps those to a wind speed; a separate classifier gives probabilities over the IMD intensity scale; two quantile models give a prediction interval.

That feature set is the Dvorak technique expressed numerically. It is also why the model trains on a laptop CPU in minutes rather than needing a GPU.

Q. What is it trained on?

The **NASA / Radiant Earth Tropical Cyclone Wind Estimation** dataset — 70,257 labelled geostationary infrared frames across 494 storms, each paired with a best-track wind speed. Public, CC-BY-4.0, no credentials.

The split is **by storm, never by frame**. Consecutive frames of one cyclone are nearly identical, so a random frame split would leak the answer across the split and report a score that is not real. A further 15% of storms are held back purely to calibrate the prediction interval.

Q. How accurate is it?

Held-out wind MAE is **10.25 kt** against a climatology baseline of 21.34 kt — a skill of **51.9%**. Exact IMD category is 43.6%; within one category is **85.3%**. For scale, the spread between trained human Dvorak analysts is around 10 kt.

Q. Does that average hide anything?

Yes, and this is the most important thing in the document. The headline is an average over a dataset dominated by moderate storms. Broken out by category, the model **over-reads weak systems and under-reads severe ones** — for a warning system, the dangerous direction.

Category	Frames	MAE (kt)	Bias (kt)
LPA	123	12.22	+12.22
D	2,303	7.25	+6.38
DD	1,748	8.88	+7.09
CS	4,219	8.49	+4.20
SCS	2,909	10.18	-2.02
VSCS	2,599	11.81	-4.29
ESCS	1,412	16.16	-12.24
SuCS	623	17.24	-15.70

Severe storms (≥ 90 kt) carry MAE 16.49 kt and a bias of -13.3 kt. There are two causes. **Class imbalance** is fixable and has been partly fixed: the training set holds ~11.7k moderate frames against ~1.0k of the most intense, so a least-squares fit minimises its loss by pulling everything toward the middle. Inverse-frequency sample weighting (alpha 0.5) cut severe MAE from 17.2 to 16.49 kt for 0.2 kt on the headline — a deliberate trade, not a free win.

Infrared saturation is not fixable here. Once cloud tops reach the tropopause they cannot get colder, so a 100 kt and a 140 kt eyewall look much alike in a single IR channel. This is the same ceiling the Dvorak technique has had since the 1970s, and it is why operational centres bring in passive microwave to resolve the inner core. No amount of reweighting removes it. Every estimate at VSCS and above therefore carries an explicit caveat telling the operator the reading is likely a lower bound, with the measured bias for that category attached.

Q. What does the confidence percentage actually mean?

It is the classifier's own probability for the category it chose, reduced when the regression interval is wide. It is not decorative: on the labelled test frames it read 96% where the estimate was 0.5 kt from truth, and 29% where it was 33 kt out. Low confidence means an ambiguous image, not a weak storm.

It is also load-bearing. The siren will not fire automatically below 70% confidence, so a shaky read cannot set off a physical alarm on its own.

Q. Why is the confidence 64% but the category probability 88%?

They answer different questions. The category probability is how much of the classifier's belief sits in that one band. The confidence is that number after a penalty for how wide the wind interval is. A storm can sit clearly inside one category while its exact wind speed remains uncertain — the first number stays high, the second comes down.

Q. What happens if I upload something that is not a cyclone?

It is refused rather than scored. Three independent gates run, and any one of them refuses the image: a **chroma check** (the model reads single-channel infrared, so colour imagery is the wrong modality), **Mahalanobis distance plus an isolation forest** (frames far from, or in a hole inside, the training distribution), and a **texture envelope** (real cloud imagery occupies a narrow band of local roughness; noise is far too rough and a synthetic gradient far too smooth). False-positive rate on genuine held-out frames is 0.60%.

The interface distinguishes “**cannot assess this image**” from “**no cyclonic system detected**”. Conflating those is how a real storm gets waved through, so when an image is refused the estimate is stripped from the response entirely — there is no number left for anyone to read by mistake.

Q. Why not a deep neural network — a CNN or a vision transformer?

Three reasons, and none of them is that it would work badly. First, the physics is already known: Dvorak tells us *which* structures matter, so encoding them directly costs 108 features instead of millions of parameters. Second, the honest constraint — around 500 storms is a small dataset for a deep model, and by-storm splitting shrinks it further. Third, a meteorologist can interrogate a feature named `eye_contrast`; they cannot interrogate filter 47 of layer 3, and this is a domain where a forecaster must be able to disagree with the machine.

A CNN is the right next step once INSAT data lands and the dataset grows.

4 · Track and intensity forecasting

Q. How does the forecast work?

A CLIPER-style approach — Climatology and Persistence, the standard baseline every operational forecast is measured against. From the storm's recent history (position, intensity, pressure, and how each has changed over the past 6, 12 and 24 hours) gradient-boosted models predict the change in latitude, longitude and wind at five lead times: 6, 12, 24, 48 and 72 hours.

The split is chronological, not random: train on earlier seasons, test on later ones. That is the only split that mirrors how the system would actually be used.

Q. How good is it, honestly?

Measured against persistence — assuming the storm keeps doing what it is doing, which is a genuinely hard baseline at short range. The result is that skill grows with lead time, which is the expected and useful shape: persistence is excellent at 6 hours and poor at 3 days.

Lead	Cases	Track error	Persistence	Skill	Intensity MAE
6 h	878	41.9 km	43.5 km	+3.8%	3.62 kt
12 h	812	83.6 km	83.5 km	-0.2%	5.88 kt
24 h	697	167.1 km	180.2 km	+7.2%	8.93 kt
48 h	515	344.2 km	404.3 km	+14.9%	15.63 kt
72 h	371	496.8 km	638.4 km	+22.2%	19.03 kt

State the weak row before anyone finds it. At 12 hours the model ties persistence — skill is -0.2% , which is zero. It only pulls clearly ahead from 24 hours ($+7.2\%$) and is strongest at 72 hours ($+22.2\%$). That is what a track-history-only model should do, and it is exactly where the next improvement lies: the model currently sees no environmental information at all. Adding sea-surface temperature, vertical wind shear and steering flow from ERA5 or GFS is the single largest accuracy gain still available anywhere in this project.

5 · The alert system and the siren tower

Q. How does alerting decide what to send?

A rule engine reads thresholds from YAML, so a meteorologist can tune them without touching code and reload them live. Severity is a function of intensity, proximity, time to landfall and confidence — never intensity alone. A Super Cyclonic Storm heading out to sea is not a red alert.

Q. Why is SMS manual but the siren automatic? Is that not inconsistent?

It is deliberate, and the difference is the point. SMS costs credits per message and goes to real people, so an unreviewed model output must not be able to text the public — an analyst reads the drafted message and presses send.

A siren costs nothing per sounding, and a tower that waits for an operator is no use at three in the morning when the link to the control room is down. So the siren may fire itself — but only above SCS, only above 70% model confidence, only when the image was actually scored, and always with a hard duration cap so a dropped link cannot leave it wailing.

Q. What is the hardware, and what does it prove?

An ESP32 microcontroller driving a three-lamp signal stack and a piezo siren, connected to the control-room machine over USB. Green is all clear; yellow with a periodic chirp is SCS; a blinking amber with a pulsed siren is VSCS; red with a continuous rising and falling wail is ESCS or above. The wail is a frequency sweep rather than a fixed tone — a steady beep reads as an appliance, a sweep reads as an emergency and carries further over wind and surf.

It also keeps working alone: if the tower is disturbed while the link to the control room is down, it raises a local alert by itself. That is what makes it a tower rather than a lamp on a wire.

Q. USB is not “no network coverage”. Is this not cheating?

Correct, and it should be said plainly rather than glossed. USB is standing in for the radio hop. The real last mile is **LoRa at 433 MHz**: five to ten kilometres on flat coast with a simple whip antenna, more from a mast, and no infrastructure of any kind between the two ends.

The command protocol was written as plain text specifically so that swap costs nothing on the software side — the control-room code is identical whether the bytes arrive over a cable or over the air. The stretch of coast that loses cellular coverage in the hours before landfall is exactly the population that most needs warning, and it is the case SMS structurally cannot serve.

6 · Architecture and how to run it

Four services, deliberately separate so that a failure in one does not silence the others. During a cyclone, network conditions are precisely when delivery paths start dropping.

Service	Port	Responsibility
Frontend (React + Vite)	5173	Dashboard, map, image analysis, alert console
Backend (FastAPI)	8000	API gateway, event source, imagery proxy, wind field
AI model service (FastAPI)	8001	Intensity from imagery, track and intensity forecasting
Alert service (FastAPI)	8002	Rule engine, message rendering, SMS, siren tower

The alert service is separate from the backend on purpose: warnings must keep flowing even if the dashboard or the model service is down.

Q. Why keep a mock path in the code at all?

So the interface stays usable while a service restarts, instead of showing an error page. Anything it produces is tagged mock: `true` and the interface renders a MOCK RESULT badge, so a generated number can never be mistaken for a predicted one. The real model is always tried first.

7 · Limitations, stated plainly

Every serious system has these. Listing them is not a weakness in a submission; being unable to list them is.

Limitation	Detail and current status
Not INSAT data	Trained on geostationary IR from another platform. Same measurement, different sensor. MOSDAC credentials applied for, not yet granted. Expect a domain gap until retrained.
Severe storms under-read	Bias -13.3 kt at ≥ 90 kt. Partly a data imbalance (mitigated), partly infrared saturation (a physical ceiling). Surfaced on every affected estimate.
No cyclone localisation	The model assumes a roughly storm-centred frame, as its training frames were. Finding the storm in a full-disk image is a separate detection problem, not built.
Pattern type is inferred	Not a trained classifier — the datasets carry wind speed, not Dvorak pattern labels. Always reported with its source so it cannot be mistaken for one.
Track model sees no environment	Track history only. Ties persistence at 12 h. ERA5/GFS predictors are the next step.
No geofenced delivery	Rules and rendering are real; resolving which recipients sit inside a warning polygon is not built.
Interval coverage slightly under target	Conformal calibration delivers 78.2% against an 80% target — honest, and reported rather than rounded up.

8 · Harder questions, and straight answers

Q. Is this not just the Dvorak technique with extra steps?

It is Dvorak's insight — that cloud structure encodes intensity — made automatic, reproducible and quantified. What changes is not the physics but three things Dvorak cannot offer: it is deterministic where manual Dvorak is subjective and varies between analysts; it produces a calibrated uncertainty interval rather than a single number; and it runs in under a second on every frame rather than requiring analyst time. The stated problem is that the current method is subjective, slow and inconsistent. Those are precisely the three properties addressed.

Q. An IMD analyst is more accurate than 10 kt. Why would they use this?

They would not use it to replace their judgement. Two honest use cases: triage — every frame gets an immediate estimate so an analyst's attention goes where it is needed; and consistency — the same image always yields the same number, which matters for verification and for post-event review. The interval and the confidence exist so a forecaster can see instantly when the model is unsure and should be overruled.

Q. How do I know these numbers are not overfitted?

Three specific defences, each of which addresses a way this normally goes wrong. Splits are **by storm**, so near-identical consecutive frames cannot straddle the split. The forecasting split is **chronological**, mirroring real use. And the prediction interval is calibrated on a **third split** held back for that purpose alone — using the test set to calibrate would be self-fulfilling.

Everything reported here is measured on storms the model has never seen, and a test fails the build if the map ever shows a storm from the training window.

Q. What would you do next, with more time?

In order of value. First, environmental predictors (ERA5 sea-surface temperature, wind shear, steering flow) for the track model — the largest single gain available. Second, INSAT retraining once MOSDAC access arrives, which closes the sensor gap. Third, passive microwave to break the infrared saturation ceiling on severe storms. Fourth, the LoRa link to make the siren tower genuinely infrastructure-free. Deep learning on imagery comes after the dataset grows, not before.

Every figure in this document was read from the trained model reports when it was generated on 10 September 2026 at 04:19 UTC. Rebuild with `python scripts/build_qa_pdf.py` after any retraining and the numbers follow automatically.